

Vite

Vite is a modern frontend build tool and development server created by Evan You (the creator of Vue). It is designed to significantly improve the speed and efficiency of web development, particularly for JavaScript-based applications.

Vite is both:

1. **A development server** optimized for fast startup and instant updates
2. **A production build tool** that bundles your application for deployment

It supports modern frameworks such as **Vue, React, Svelte, Solid**, and also works with **vanilla JavaScript/TypeScript**.

The Core Problem Vite Solves

Traditional tools (e.g., Webpack) bundle the *entire application* before starting the dev server. As projects grow, this leads to:

- Slow startup times
- Sluggish hot reloads
- Increasing developer friction

Vite takes a fundamentally different approach.

How Vite Improves Web Development

1. Near-Instant Dev Server Startup

- Vite **does not bundle your app during development**
- It serves source files directly using **native ES modules (ESM)** in the browser
- Only the files actually requested by the browser are processed

Result: Dev servers start in milliseconds, even for large projects.

2. Extremely Fast Hot Module Replacement (HMR)

- When you edit a file, Vite updates **only that module**
- No full page reloads
- No rebuilding of unrelated dependencies

Result: Updates feel instantaneous, improving developer feedback loops.

3. Pre-Bundling Dependencies with esbuild

- Third-party dependencies (node_modules) are pre-bundled once using **esbuild**

- esbuild is written in Go and is **orders of magnitude faster** than JavaScript-based bundlers

Result: Faster cold starts and consistent module resolution.

4. Efficient Production Builds

For production, Vite uses **Rollup**, a mature and highly optimized bundler:

- Tree-shaking for smaller bundles
- Code splitting
- Optimized asset handling (CSS, images, fonts)

Result: Fast development without sacrificing production performance.

5. First-Class Modern JavaScript Support

Vite assumes modern browser capabilities:

- Native ES modules
- Dynamic imports
- TypeScript, JSX, and CSS preprocessors out of the box

This reduces configuration complexity and boilerplate.

6. Simple and Predictable Configuration

- Minimal default configuration
- Clear plugin system
- Shared configuration between dev and build phases

Compared to Webpack, Vite configs are typically **shorter, easier to read, and easier to maintain**.

Summary

Benefit	Impact
Instant startup	No waiting for bundling
Fast HMR	Immediate visual feedback
Modern defaults	Less configuration
Efficient builds	Optimized production output
Framework agnostic	Works with most frontend stacks

When Vite Is the Right Choice

Vite is particularly well-suited for:

- Modern frontend applications
- Projects using React, Vue, or similar frameworks
- Teams prioritizing developer experience and iteration speed

It may be less suitable if you must support very old browsers without transpilation or rely heavily on legacy tooling.

Components

Components are the building blocks of React applications. They let us split the UI into independent, reusable pieces, and think about each piece in isolation.

1. Example Component

```
import { useState, useEffect } from "react";

function UserGreeting({ name }) {
  const [message, setMessage] = useState("");

  useEffect(() => {
    setMessage(`Welcome, ${name}!`);
  }, [name]);

  return (
    <div>
      <h1>{message}</h1>
      <button onClick={() => setMessage("Hello again!")}>
        Update Message
      </button>
    </div>
  );
}

export default UserGreeting;
```

Let's break this down piece by piece.

2. Imports

```
import { useState, useEffect } from "react";
```

- **useState**: Manages local component state
- **useEffect**: Handles side effects (data fetching, subscriptions, DOM updates)

Hooks replace most use cases of class lifecycle methods.

3. Component Definition

```
function UserGreeting({ name }) {
```

- This is a **functional component**
- It receives **props** as an argument
- Props are **read-only inputs** passed from a parent component

Equivalent conceptually to parameters in a function.

4. State Management (**useState**)

```
const [message, setMessage] = useState("");
```

- **message**: Current state value
- **setMessage**: Function to update state
- **useState(" ")**: Initial state value

Key characteristics:

- State updates trigger a re-render
- State is scoped to the component instance

5. Side Effects (**useEffect**)

```
useEffect(() => {  
  setMessage(`Welcome, ${name}!`);  
}, [name]);
```

- Runs **after render**
- Dependency array **[name]** means:
 - Effect runs on initial render
 - Runs again when **name** changes

Common use cases:

- Fetching data
- Syncing props to state
- Subscribing/unsubscribing to external systems

6. JSX (Render Output)

```
return (  
  <div>  
    <h1>{message}</h1>  
    <button onClick={() => setMessage("Hello again!")}>  
      Update Message  
    </button>  
  </div>  
);
```

JSX is:

- A syntax extension for JavaScript
- Compiled into `React.createElement()` calls
- Allows embedding expressions with `{ }`

Important rules:

- Must return a single root element
- JSX can contain logic, but heavy logic should live outside the return

7. Event Handling

```
<button onClick={() => setMessage("Hello again!")}>
```

- React uses **camelCase** event handlers
- You pass a function reference, not a string
- State updates are batched and efficient

8. Export

```
export default UserGreeting;
```

- Makes the component available to other modules
- Default export is common for React components

9. How This Component Behaves at Runtime

1. Component mounts
2. `useState` initializes `message`
3. `useEffect` runs and sets the welcome message
4. JSX renders to the DOM
5. Clicking the button updates state
6. Component re-renders with new state

10. Key Concepts Summarized

Concept	Purpose
Component	Encapsulated UI + behavior
Props	External, read-only inputs
State	Internal, mutable data
Hooks	Add behavior to functions
JSX	Declarative UI syntax
Re-render	Triggered by state/prop changes

Functional Components

Functional components are some of the more common components that will come across while working in React. These are simply JavaScript functions. We can create a functional component to React by writing a JavaScript function. These functions may or may not receive data as parameters. In the functional Components, the return value is the JSX code to render to the DOM tree. Functional components can also have state which is managed using React hooks.

Below is a structured breakdown of a **React component**, focusing on a modern **functional component** using hooks, which is the current standard in React development.

JSX

JSX (JavaScript XML) is a syntax extension for JavaScript used by React to describe **UI structure**.

It lets you write code that *looks like HTML* but is actually **JavaScript expressions**.

```
const element = <h1>Hello, world</h1>;
```

JSX is **not HTML** and **not valid JavaScript by itself**.

What JSX Compiles To

JSX is transformed at build time into plain JavaScript:

```
<h1>Hello</h1>
```

becomes:

```
React.createElement("h1", null, "Hello");
```

With modern React (17+), this is handled automatically via the **new JSX transform**, so you no longer need to import React explicitly.

Why JSX

JSX improves:

- **Readability** – UI structure is visually clear
- **Maintainability** – UI and logic live together
- **Declarative programming** – You describe *what* the UI should be, not *how* to manipulate the DOM

Without JSX, React code becomes verbose and harder to reason about.

JSX Is Just Expressions

Everything inside JSX must resolve to a **JavaScript expression**.

```
<h1>{user.name}</h1>
```

```
<p>{count + 1}</p>
```

Not allowed:

- Statements (*if*, *for*, *while*)
- Assignments

Instead, use:

- Ternaries
- Logical operators
- Array methods (*map*)

JSX Rules That Matter

1. Single Root Element

```
return (  
  <>  
    <Header />  
    <Content />  
  </>  
);
```

Fragments (*<> . . . </>*) avoid extra DOM nodes.

2. JavaScript Naming Conventions

- `class` → `className`
- `for` → `htmlFor`

Event handlers use camelCase:

`onClick={handleClick}`

3. Components Must Be Capitalized

`<MyComponent />` // React component

`<div />` // HTML element

Lowercase names are treated as native DOM tags.

JSX and Security

JSX **automatically escapes values** to prevent XSS attacks:

`<p>{userInput}</p>`

Raw HTML must be explicitly injected:

`<div dangerouslySetInnerHTML={{ __html: html }} />`

This is intentionally verbose to discourage misuse.

Summary

JSX is a declarative syntax that compiles to JavaScript function calls, allowing React to describe UI as data instead of manipulating the DOM directly.

Props vs State

Props (short for “properties”) and state are both plain JavaScript objects. While both hold information that influences the output of component render, they are different in one important way: props get passed to the component (similar to function parameters) whereas state is managed within the component (similar to variables declared within a function).

Below is a **short, clear, but technically correct** explanation of **Props vs State** and **how to use each in React**.

1. What Are Props?

Props (properties) are **inputs** passed *into* a component from its parent.

- Read-only
- Controlled by the parent
- Used to configure or display data

Example: Using Props

```
function Greeting({ name }) {  
  return <h1>Hello, {name}</h1>;  
}
```

```
<Greeting name="Alice" />
```

Key point: The component cannot modify **name**.

2. What Is State?

State is **internal, mutable data** owned by the component.

- Managed inside the component
- Can change over time
- Changes trigger re-renders

Example: Using State

```
import { useState } from "react";
```

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <button onClick={() => setCount(count + 1)}>  
      Count: {count}  
    </button>  
  );  
}
```

3. Core Differences (Props vs State)

Aspect	Props	State
Ownership	Parent	Component
Mutability	Immutable	Mutable

Purpose	Configure / pass data	Track changing data
Updates	Parent re-renders	<code>setState</code> / <code>setX</code>
Scope	External	Internal

- **Props** = “data I receive”
- **State** = “data I manage”

4. How They Work Together

A common pattern:

1. Parent holds **state**
2. Parent passes state down as **props**
3. Child triggers updates via **callbacks**

Example: Lifting State Up

```
function Parent() {
  const [count, setCount] = useState(0);

  return <Child count={count} onIncrement={() => setCount(count + 1)} />;
}

function Child({ count, onIncrement }) {
  return <button onClick={onIncrement}>{count}</button>;
}
```

This keeps data flow **unidirectional** (top → down).

5. When to Use Props vs State

Use Props When:

- Data comes from a parent
- Component is presentational
- You want reusability and predictability

Use State When:

- Data changes over time
- Component handles user interaction
- Component needs to remember something

6. Common Mistakes to Avoid

Copying props into state unnecessarily

```
// Usually wrong
const [value, setValue] = useState(props.value);
```

Instead, derive directly from props unless you need local control.

Mutating state directly

```
// Wrong
count++;
```

Always use the setter:

```
setCount(prev => prev + 1);
```

Conditional Rendering

In React, you can create distinct components that encapsulate behavior you need. Then, you can render only some of them, depending on the state of your application.

Conditional rendering in React works the same way conditions work in JavaScript. Use JavaScript operators like `if` or the conditional operator to create elements representing the current state, and let React update the UI to match them.

Conditional rendering means showing **different UI** based on a condition (state, props, or derived values).

React does not have special syntax for this—you use **JavaScript**.

1. `if` Statements (Outside JSX)

Use when logic is complex.

```
function Status({ isLoggedIn }) {
  if (isLoggedIn) {
    return <Dashboard />;
  }

  return <Login />;
}
```

Best for:

- Early returns
- Clear branching logic

2. Ternary Operator

Use inside JSX for simple conditions.

```
{isLoggedIn ? <Dashboard /> : <Login />}
```

Readable and concise when kept short.

3. Logical AND (&&)

Render something **only if** a condition is true.

```
{isAdmin && <AdminPanel />}
```

Important caveat:

- `0`, `" "`, or `false` will render unintentionally
- Prefer explicit booleans

4. Conditional Attributes and Content

You can conditionally set props and text.

```
<button disabled={isLoading}>  
  {isLoading ? "Loading..." : "Submit"}  
</button>
```

This is a very common UI pattern.

5. Conditional Rendering with Functions

For reuse and clarity.

```
function renderMessage() {  
  if (error) return <Error />;  
  if (loading) return <Spinner />;  
  return <Content />;  
}  
  
return <div>{renderMessage()}</div>;
```

6. Rendering Lists Conditionally

```
{items.length > 0 ? (  
  <ItemList items={items} />  
) : (  
  <EmptyState />  
)}
```

Avoid rendering empty containers.

7. Conditional Rendering vs CSS Hiding

Conditional Rendering (Preferred)

```
{isOpen && <Modal />}
```

- Component mounts/unmounts
- Effects run correctly
- Better performance

CSS Hiding

```
<div style={{ display: isOpen ? "block" : "none" }} />
```

- Component still exists in DOM
- Effects still run

8. Best Practices

- Keep conditions **simple and readable**
- Avoid deeply nested ternaries
- Extract logic into variables or functions
- Prefer early returns for complex flows

One-Sentence Rule

If UI changes based on data, conditional rendering is the correct tool—not DOM manipulation.

Composition vs Inheritance

React has a powerful composition model, and it is recommended to use composition instead of inheritance to reuse code between components.

What Is Composition?

Composition means **building components by combining smaller components** instead of extending them.

In React, composition is achieved through:

- Props
- `children`
- Render props (less common now)
- Custom hooks

Example: Basic Composition

```
function Card({ children }) {  
  return <div className="card">{children}</div>;  
}
```

```
function Profile() {  
  return (  
    <Card>  
      <h2>User Profile</h2>  
      <p>Account details here</p>  
    </Card>  
  );  
}
```

Here:

- `Card` is a **generic container**
- `Profile` injects content via `children`

No inheritance is involved.

Specialization via Props (Not Inheritance)

```
function Button({ variant, children }) {  
  return (  
    <button className={`btn btn-${variant}`}>  
      {children}  
    </button>  
  );  
}
```

```
}
```

```
<Button variant="primary">Save</Button>  
<Button variant="danger">Delete</Button>
```

One component, many behaviors—no subclassing required.

What Is Inheritance?

Inheritance means extending a base class.

```
class Button extends React.Component {}  
class PrimaryButton extends Button {}
```

Why This Is Discouraged in React

- Creates tight coupling
- Harder to refactor
- Poor fit for UI variation
- Breaks React's data flow model

React's documentation explicitly advises against this pattern.

Why Composition

1. Better Flexibility

Composition allows arbitrary combinations:

```
<Layout>  
  <Sidebar />  
  <MainContent />  
</Layout>
```

Inheritance enforces rigid hierarchies.

2. Clear Data Flow

- Data flows via props
- Behavior is explicit
- No hidden parent logic

Inheritance obscures behavior inside base classes.

3. Easier Reuse

You reuse **behavior**, not **structure**.

- Custom hooks reuse logic
- Components reuse UI
- Both compose cleanly

Composition with Custom Hooks (Logic Reuse)

```
function useToggle() {  
  const [value, setValue] = useState(false);  
  return [value, () => setValue(v => !v)];  
}
```

```
const [isOpen, toggle] = useToggle();
```

This replaces inheritance-based logic sharing.

Mental Model

- **Inheritance**: “What is this?”
- **Composition**: “What does this use?”

React answers UI problems with **what it uses**, not **what it extends**.

Rendering

React follows a declarative approach to rendering components, which means that developers specify what a component should look like, and React takes care of rendering the component to the screen. This is in contrast to an imperative approach, where developers would write code to manually manipulate the DOM (Document Object Model) to update the UI.

The virtual DOM (VDOM) is an important aspect of how React works. It is a lightweight in-memory representation of the DOM (Document Object Model), and it is used to optimize the rendering of components in a React application.

- Components are written as JavaScript classes or functions that define a render method. The render method returns a description of what the component should look like, using JSX syntax.
- When a component is rendered, React creates a virtual DOM (VDOM) representation of the component. The VDOM is a lightweight in-memory representation of the DOM, and it is used to optimize the rendering of components.
- React compares the VDOM representation of the component with the previous VDOM representation (if it exists). If there are differences between the two VDOMs, React calculates the minimum number of DOM updates needed to bring the actual DOM into line with the new VDOM.

- React updates the actual DOM with the minimum number of DOM updates needed to reflect the changes in the VDOM.

This process is known as reconciliation, and it is an important aspect of how React works. By using a declarative approach and a VDOM, React is able to optimize the rendering of components and improve the performance of web applications.

React describes UI updates in **three distinct phases**:

1. **Triggering a render**
2. **Rendering the component**
3. **Committing to the DOM**

1. Triggering a Render

(Delivering the guest's order to the kitchen)

A render is triggered when React is told **something changed**.

What triggers it:

- State updates (`setState`, `setCount`)
- New props from a parent
- Context value changes

```
setCount(count + 1);
```

At this point:

- Nothing is rendered yet
- The DOM is untouched
- React just queues the update

Analogy:

The waiter delivers the order to the kitchen. No food is prepared yet—React just knows *work is needed*.

2. Rendering the Component

(Preparing the order in the kitchen)

React now **calls your component functions** to calculate what the UI *should look like*.

What happens here:

- Components run as pure functions

- JSX is evaluated
- A new **virtual DOM tree** is created
- React compares it with the previous tree (**reconciliation**)

```
function Counter({ count }) {
  return <h1>{count}</h1>;
}
```

Important characteristics:

- No DOM changes yet
- No side effects should occur here
- This phase can be paused, restarted, or discarded (especially in Concurrent React)

Analogy:

The kitchen prepares the dish. The chef figures out *what to cook*, but nothing reaches the table yet.

3. Committing to the DOM

(Placing the order on the table)

Once React knows **exactly what changed**, it applies the **minimal set of DOM updates**.

What happens here:

- DOM nodes are created, updated, or removed
- `useLayoutEffect` runs
- Browser paints the updated UI
- Then `useEffect` runs

```
useEffect(() => {
  console.log("Rendered to screen");
});
```

This is the only phase where:

- The user sees changes
- The browser is affected

Analogy:

The prepared dish is placed on the table. The guest can finally see and interact with it.

Why This Separation Matters

Predictability

- Rendering is pure and repeatable
- DOM mutation is controlled and minimal

Performance

- React batches updates
- Avoids unnecessary DOM work

Concurrency

- React can interrupt or restart rendering
- Commit phase remains fast and atomic

One Diagram Summary

State/Props change



Trigger render



Render phase (calculate UI)



Commit phase (update DOM)

Key Rules to Remember

- **State updates trigger renders**
- **Rendering is calculation, not mutation**
- **Only the commit phase touches the DOM**
- **Effects run after commit**

One-Line

React first decides *what* the UI should be, then commits *only what changed* to the DOM.

Component Life Cycle

React components have a lifecycle consisting of three phases: Mounting, Updating, and Unmounting along with several “lifecycle methods” that you can override to run code at particular times in the process.

It is not recommended to use lifecycle methods manually. Instead, use the `useEffect` hook with functional components.

The **three main phases**:

1. **Mounting** – component is added to the screen
2. **Updating** – component re-renders due to changes
3. **Unmounting** – component is removed from the screen

These phases map directly to **effects and rendering behavior** in modern React.

1. Mounting Phase

(Component appears on the screen)

What happens:

- Component function runs for the first time
- JSX is calculated
- DOM is created and committed
- Effects run

Hooks involved:

```
useEffect(() => {  
  // Runs after first render  
}, []);
```

```
useLayoutEffect(() => {  
  // Runs before browser paint  
}, []);
```

Lifecycle meaning:

- Initialization logic
- Data fetching
- Subscriptions

Equivalent (class components):

- `componentDidMount`

2. Updating Phase

(Component re-renders due to change)

Triggered by:

- State changes
- Prop changes
- Context updates

What happens:

1. Render phase recalculates UI
2. React diffs old vs new output
3. DOM updates are committed
4. Effects re-run if dependencies changed

Hooks involved:

```
useEffect(() => {
  // Runs when `value` changes
}, [value]);
```

Lifecycle meaning:

- Syncing UI with data
- Responding to user interactions

Equivalent (class components):

- `componentDidUpdate`

3. Unmounting Phase

(Component is removed from the screen)

What happens:

- Component is removed from the DOM
- Cleanup logic runs

Cleanup with Hooks:

```
useEffect(() => {
  const id = setInterval(doSomething, 1000);

  return () => {
    clearInterval(id); // cleanup
  };
}, []);
```

Lifecycle meaning:

- Cancel timers
- Remove event listeners
- Clean up subscriptions

Equivalent (class components):

- `componentWillUnmount`

Lifecycle with Hooks vs Classes

Lifecycle Phase	Hooks	Class Components
Mount	<code>useEffect(..., [])</code>	<code>componentDidMount</code>
Update	<code>useEffect(..., [deps])</code>	<code>componentDidUpdate</code>
Unmount	Cleanup function	<code>componentWillUnmount</code>

Important Clarifications

Rendering ≠ Lifecycle Methods

- Rendering is **calculation**
- Effects are **side effects**
- Lifecycle logic belongs in effects, not render

Effects Run After Commit

- DOM must exist before effects run
- React guarantees timing

Rendering decides what the UI should look like.
Effects synchronize the outside world with that UI.

Lifecycle Timeline

Mount

```

|— Render
|— Commit DOM
|— Run Effects

```

Update

```

|— Render
|— Commit DOM changes
|— Run Effects (if deps changed)

```

Unmount

```

|— Cleanup Effects

```

In Short line

In modern React, lifecycle behavior is expressed through **useEffect** and its dependency array, not explicit lifecycle methods.

Lists and Keys

When you render lists in React, you can use the **key** prop to specify a unique key for each item. This key is used to identify which item to update when you want to update a specific item.

What Are Lists in React?

Lists are created by **mapping data to components**.

```
const items = ["Apple", "Banana", "Orange"];
```

```
return (  
  <ul>  
    {items.map(item => (  
      <li>{item}</li>  
    ))}  
  </ul>  
);
```

This renders multiple elements from an array.

What Is a Key?

A **key** is a **special prop** that helps React **identify which list item corresponds to which component** between renders.

```
<li key={item.id}>{item.name}</li>
```

Keys are **not passed to the component** as props.

Why Keys Matter

React uses keys during **reconciliation** to decide:

- Which items stayed the same
- Which items changed
- Which items were added or removed

Without keys, React falls back to **array index comparison**, which leads to bugs.

Correct Way to Render Lists

```
const users = [
  { id: 1, name: "Alice" },
  { id: 2, name: "Bob" }
];

return (
  <ul>
    {users.map(user => (
      <li key={user.id}>{user.name}</li>
    ))}
  </ul>
);
```

Key rules:

- Keys must be **stable**
- Keys must be **unique among siblings**
- Keys should come from your data

Common Mistake: Using Index as Key

```
{items.map((item, index) => (
  <li key={index}>{item}</li>
))}
```

Why this is dangerous:

- Reordering breaks identity
- State moves to the wrong component
- Causes subtle UI bugs

Only acceptable if:

- List is static
- Items never reorder, insert, or delete

Keys and Component State

```
<input />
```

If keys change incorrectly:

- React reuses DOM nodes
- Input values appear to “jump”
- State leaks between items

Keys define **component identity**.

Keys Are for React, Not You

```
function Item({ id }) {
  // `key` is NOT accessible here
}
```

If you need the ID:

```
<Item key={item.id} id={item.id} />
```

Nested Lists

Keys apply to the **direct children being iterated**.

```
sections.map(section => (
  <Section key={section.id}>
    {section.items.map(item => (
      <Item key={item.id} />
    ))}
  </Section>
));
```

One-Liner

Keys tell React “this is the same thing as before” so state and DOM are preserved correctly.

Check that

- Stable ID from data?
- Unique among siblings?
- No index unless static list?

One-Line Rule

If a list item can move, be removed, or added—do not use the array index as a key.

Render Props

The term 'render props' refers to a technique for sharing code between React components using a prop whose value is a function.

A component with a render prop takes a function that returns a React element and calls it instead of implementing its own render logic.

A **render prop** is a **function prop** that a component uses to decide **what to render**.

The term “render prop” refers to a pattern, not a React feature.

```
<DataProvider render={data => <Display data={data} />} />
```

The component delegates rendering to the function it receives.

Why Render Props

Render props solve **logic reuse without inheritance**.

They allow:

- Sharing stateful logic
- Full control over rendering
- Avoiding tight coupling between components

This pattern predates hooks.

Basic Example

Component with Shared Logic

```
function MouseTracker({ render }) {  
  const [position, setPosition] = useState({ x: 0, y: 0 });  
  
  return (  
    <div onMouseMove={e => setPosition({ x: e.clientX, y: e.clientY })}>  
      {render(position)}  
    </div>  
  );  
}
```

Consumer Controls Rendering

```
<MouseTracker  
  render={({ x, y }) => (  
    <h1>Mouse position: {x}, {y}</h1>  
  )  
>
```

```
  })  
/>
```

Here:

- **MouseTracker** manages logic
- The consumer decides the UI

children as a Render Prop

A common variation:

```
function DataFetcher({ children }) {  
  const data = useData();  
  return children(data);  
}
```

```
<DataFetcher>  
  {data => <Chart data={data} />}  
</DataFetcher>
```

This feels more natural in JSX.

How Render Props Work in the Lifecycle

- Render prop function is called **during render**
- It must be **pure**
- No side effects inside the render function

React treats it like any other component render.

Downsides of Render Props

1. JSX Nesting

```
<A>  
  {a => (  
    <B>  
      {b => (  
        <C>  
          {c => <View a={a} b={b} c={c} />}  
        </C>  
      )}  
    </B>  
  )}  
</A>
```


Hard to read and maintain.

2. Performance Considerations

- New function created on each render
- Can prevent memoization unless optimized

Usually not an issue, but worth noting.

Render Props vs Hooks

Render Props	Hooks
Share logic via components	Share logic via functions
JSX-heavy	Cleaner call sites
Pre-hooks pattern	Preferred today

Equivalent with Hooks

```
function useMouse() {  
  const [pos, setPos] = useState({ x: 0, y: 0 });  
  return pos;  
}
```

```
const { x, y } = useMouse();
```

Hooks replaced most render prop use cases.

When Render Props Still Make Sense

- Library components needing **UI flexibility**
- Cross-cutting concerns (measurement, positioning)
- When hooks cannot be used (class components)
- When you want explicit rendering control

Examples:

- Tooltips
- Modals
- Virtualized lists

One-Liner

Render props let a component share logic while letting the consumer decide the UI.

Rule

If you're writing application code, prefer hooks; if you're writing reusable UI primitives, render props may still be appropriate.

Refs

Refs provide a way to access DOM nodes or React elements created in the render method.

In the typical React dataflow, props are the only way that parent components interact with their children. To modify a child, you re-render it with new props. However, there are a few cases where you need to imperatively modify a child outside of the typical dataflow. The child to be modified could be an instance of a React component, or it could be a DOM element. For both of these cases, React provides an escape hatch.

A **ref** is an object that lets you **hold a mutable value** that **persists across renders without triggering a re-render**.

```
const ref = useRef(initialValue);
```

Refs are commonly used to:

- Access DOM elements
- Store mutable values (timeouts, previous values, external APIs)

1. How to Add a Ref to Your Component

Creating a Ref

```
import { useRef } from "react";

function InputFocus() {
  const inputRef = useRef(null);

  return <input ref={inputRef} />;
}
```

- `useRef(null)` creates a ref object
- React assigns the DOM node to `ref.current` after commit

Accessing the DOM Element

```
inputRef.current.focus();
```

This runs **after the component is mounted**.

2. How to Update a Ref's Value

Refs are **mutable**.

```
const countRef = useRef(0);
```

```
countRef.current += 1;
```

Key points:

- Updating `.current` does **not** trigger a render
- Value persists between renders

This makes refs ideal for storing:

- Timers
- External library instances
- Previous values

3. How Refs Are Different from State

Aspect	Refs	State
Triggers re-render	No	Yes
Mutability	Mutable	Immutable
UI updates	No	Yes
Use case	Escape hatch	UI data
Lifecycle	Persists	Persists

Rule of Thumb

- **State** → affects what you render
- **Refs** → do not affect rendering

4. How to Use Refs Safely

Use Refs For

- DOM access (`focus`, `scroll`, `measure`)
- Storing mutable values that shouldn't re-render
- Integrating non-React libraries

```
const timerRef = useRef(null);
```

Do NOT Use Refs For

- Data that should appear in the UI
- Replacing state logic
- Controlling rendering behavior

// Anti-pattern

```
ref.current = newValue; // UI won't update
```

Use Refs Only After Commit

DOM refs are populated **after render**.

Correct usage:

```
useEffect(() => {
  inputRef.current.focus();
}, []);
```

Forwarding Refs

By default, refs **do not pass through components**.

```
const Input = forwardRef((props, ref) => {
  return <input ref={ref} />;
});
```

Used in:

- Design systems
- Reusable input components

Strict Mode Behavior

In development:

- Components may mount twice
- Refs may be set and cleared twice

Your code must tolerate this.

One-Line Summary

Refs are an escape hatch from React's one-way data flow.

Use them sparingly and intentionally.

Refs let you store mutable values and access DOM nodes without causing re-renders, but they should never replace state.

Events

Handling events with React elements is very similar to handling events on DOM elements. There are some syntax differences:

- React events are named using camelCase, rather than lowercase.
- With JSX you pass a function as the event handler, rather than a string.

1. Different Ways to Write an Event Handler

React events are **synthetic events** that wrap native browser events and behave consistently across browsers.

1.1 Inline Arrow Function

```
<button onClick={() => console.log("Clicked")}>  
  Click me  
</button>
```

Use when:

- Logic is simple
- No performance concern

Trade-off:

- New function created on every render

1.2 Named Function

```
function handleClick() {  
  console.log("Clicked");  
}
```

```
<button onClick={handleClick}>Click me</button>
```

Best practice:

- Cleaner
- Easier to test and reuse

1.3 Passing Arguments to Handlers

```
<button onClick={() => handleDelete(id)}>
  Delete
</button>
```

You must wrap the call in a function—do not call it directly.

1.4 Using the Event Object

```
function handleSubmit(event) {
  event.preventDefault();
}
```

React passes the event automatically.

2. How to Pass Event Handling Logic from a Parent Component

This is how React maintains **one-way data flow**.

Parent Defines the Logic

```
function Parent() {
  function handleSave(data) {
    console.log(data);
  }

  return <Child onSave={handleSave} />;
}
```

Child Triggers the Event

```
function Child({ onSave }) {
  return (
    <button onClick={() => onSave("Hello")}>
      Save
    </button>
  );
}
```

Key idea:

- Parent owns the behavior
- Child only signals intent

Common Naming Convention

Prop Name	Meaning
<code>onClick</code>	Event callback
<code>handleClick</code>	Implementation
<code>onSubmit</code>	Action signal

3. How Events Propagate and How to Stop Them

React events **bubble up**, just like DOM events.

Event Bubbling Example

```
function App() {  
  return (  
    <div onClick={() => console.log("Parent")}>  
      <button onClick={() => console.log("Child")}>  
        Click  
      </button>  
    </div>  
  );  
}
```

Clicking the button logs:

```
Child  
Parent
```

Stopping Propagation

```
function handleClick(event) {  
  event.stopPropagation();  
}
```

```
<button onClick={handleClick}>Click</button>
```

Now only the button handler runs.

Preventing Default Browser Behavior

```
function handleSubmit(event) {  
  event.preventDefault();  
}
```

Stops:

- Form submission
- Page reloads

Important Rules

- Use **camelCase** event names (`onClick`, not `onclick`)
- Pass **functions**, not strings
- Events run during the **commit phase**
- Event handlers should be fast and side-effect aware

One-Sentence Rule

Events let the user talk to your code; state changes let your code talk back to the UI.

Event handlers express intent; state changes express results.

High Order Components

A higher-order component (HOC) is an advanced technique in React for reusing component logic. HOCs are not part of the React API, per se. They are a pattern that emerges from React's compositional nature.

Concretely, a higher-order component is a function that takes a component and returns a new component.

Higher-order components are not commonly used in modern React code. In order to reuse logic, React hooks are mainly used now.

Hooks

Hooks were introduced in React 16.8 and they let us use React's features-like managing your component's state and or performing an after effect when certain changes occur in state(s) without writing a class.

Hooks are functions that let you **use React features in function components**, such as:

- State
- Lifecycle behavior
- Context
- Reusable logic

Hooks remove the need for class components in most cases.

```
const [count, setCount] = useState(0);
```

Why Hooks

Hooks solve three long-standing problems in React:

1. **Logic reuse** without HOCs or render props
2. **Complex lifecycle logic** spread across methods
3. **Class syntax complexity**

Hooks make components:

- Smaller
- More readable
- Easier to compose

Core Rules of Hooks

React enforces these rules:

1. **Only call hooks at the top level**
2. **Only call hooks from React functions**

```
// Wrong
if (condition) {
  useState();
}
```

```
// Correct
useState();
```

This ensures consistent hook ordering across renders.

Core Hooks

1. **useState** — Local State

```
const [value, setValue] = useState(initialValue);
```

- Triggers re-render on update
- Updates are asynchronous and batched

Use when:

- UI changes over time
- User interaction occurs

2. **useEffect** — Side Effects

```
useEffect(() => {  
  // side effect  
  return () => cleanup();  
}, [dependencies]);
```

Runs:

- After commit
- On mount, update, or unmount depending on deps

Use for:

- Data fetching
- Subscriptions
- DOM interactions

3. **useRef** — Mutable Values / DOM Access

```
const ref = useRef(null);
```

- Does not trigger re-render
- Persists across renders

Use for:

- DOM access
- Timers
- External libraries

4. **useContext** — Shared State

```
const value = useContext(MyContext);
```

- Avoids prop drilling
- Subscribes to context changes

5. **useMemo** — Memoize Values

```
const result = useMemo(() => compute(data), [data]);
```

- Prevents expensive recalculations
- Optimization, not logic

6. **useCallback** — Memoize Functions

```
const handleClick = useCallback(() => {  
  doSomething();  
}, []);
```

- Stabilizes function references
- Helps with memoized children

Custom Hooks

Custom hooks let you extract reusable logic.

```
function useCounter() {  
  const [count, setCount] = useState(0);  
  return { count, increment: () => setCount(c => c + 1) };  
}
```

Usage:

```
const { count, increment } = useCounter();
```

Rules:

- Must start with **use**
- Can call other hooks
- Must follow hook rules

Hooks vs Lifecycle Methods

Class Lifecycle	Hook Equivalent
constructor	<code>useState</code>
componentDidMount	<code>useEffect(..., [])</code>
componentDidUpdate	<code>useEffect(..., [deps])</code>
componentWillUnmount	Cleanup function

Hooks and Rendering

- Hooks run during **render**
- Effects run after **commit**
- Hook order must never change

Common Hook Mistakes

- Overusing `useEffect`
- Missing dependencies
- Using refs instead of state
- Premature optimization with `useMemo`

One-Sentence

Hooks let you plug React features into pure functions while keeping rendering predictable.

Hooks replace classes by making stateful logic composable and explicit.

Basic Hooks

`useState`

1. What is `useState`?

`useState` is a React Hook that allows function components to hold and manage local, mutable state across renders.

"`useState` preserves state between re-renders and triggers a re-render when the state is updated."

```
const [state, setState] = useState(initialValue);
```

2. Why `useState` Exists

Before hooks:

- State was only available in **class components**
- Functional components were stateless

With hooks:

- Functions can now:
 - Store state
 - React to changes
 - Re-render predictably

Mental model:

- State = data that changes over time
- Render = UI snapshot based on state

3. Core Rules (Very Important for Interviews)

Rule 1: Hooks must be called at the top level

 Wrong

```
if (condition) {  
  useState(0);  
}
```

 Correct

```
const [count, setCount] = useState(0);
```

Why: React relies on **call order**, not names.

Rule 2: Hooks must be called in React functions only

- Function components
- Custom hooks

4. Basic Example (Hands-On #1)

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <>  
      <p>Count: {count}</p>  
      <button onClick={() => setCount(count + 1)}>  
        Increment  
      </button>  
    </>  
  );  
}
```

What happens internally?

1. React stores `count`
2. Button click → `setCount`
3. React schedules a re-render
4. UI updates

5. Interview Trap: State Updates Are Asynchronous

```
setCount(count + 1);  
console.log(count); // NOT updated immediately
```

Correct explanation:

State updates are batched and applied during the next render cycle.

6. Functional Updates (CRITICAL for Interviews)

Problem

```
setCount(count + 1);  
setCount(count + 1); // stale value
```

Solution

```
setCount(prev => prev + 1);  
setCount(prev => prev + 1);
```

Interview phrase:

Use functional updates when the next state depends on the previous state.

7. useState with Objects (Hands-On #2)

```
const [user, setUser] = useState({  
  name: 'Alice',  
  age: 25  
});
```

 Wrong

```
user.age = 26;  
setUser(user);
```

 Correct

```
setUser(prev => ({
```

```
    ...prev,  
    age: 26  
  }));
```

Why: React compares references, not deep values.

8. useState with Arrays (Hands-On #3)

```
const [items, setItems] = useState([]);
```

Add item:

```
setItems(prev => [...prev, newItem]);
```

Remove item:

```
setItems(prev => prev.filter(item => item.id !== id));
```

9. Lazy Initialization (Advanced Interview Topic)

```
const [value, setValue] = useState(() => {  
  return expensiveCalculation();  
});
```

Why:

- Runs once on initial render
- Prevents unnecessary computation

Interview line:

Lazy initialization improves performance when computing the initial state is expensive.

10. When NOT to Use useState (Very Important)

Do NOT use state for:

- Derived values
- Props-based calculations

 Wrong

```
const [fullName, setFullName] = useState(first + last);
```

 Correct

```
const fullName = first + last;
```

11. Common Interview Questions & Answers

Q: Does `useState` merge objects like `setState` in class components?

Answer: No. It replaces the entire state value.

Q: Why does updating state trigger re-render?

Answer: React uses state changes to determine when the UI should be re-evaluated.

Q: Can multiple `useState` hooks be used?

Answer: Yes. React matches them by call order.

12. Hands-On Mini Assignment (Do This Now)

Task 1: Counter with Reset

Requirements:

- Increment
- Decrement
- Reset to 0

Task 2: Input Field

Requirements:

- Controlled input
- Display live typed value

Task 3: Todo List (Simple)

Requirements:

- Add items
- Remove items
- Use array state immutably

13. Documentation-Ready Summary

`useState` enables state management in functional components. It returns a stateful value and a setter function. State updates are asynchronous, replace the previous value, and trigger re-renders. Functional updates should be used when a new state depends on a previous state. Immutability is required for objects and arrays.

useEffect

1. Interview Definition

useEffect lets you perform side effects in function components after render.

Key phrases interviewers expect:

“**useEffect** runs after the render phase and is used for effects like data fetching, subscriptions, timers, and manual DOM interactions.”

2. Why **useEffect** Exists (Conceptual)

React has **two phases**:

1. **Render phase**—calculate UI
2. **Commit phase** – apply changes to the DOM

👉 **Side effects do NOT belong in render.**

Examples of side effects:

- API calls
- Event listeners
- Timers
- Logging
- DOM manipulation

useEffect is React’s way of saying:

“Run this logic *after* React finishes rendering.”

3. Basic Syntax (Documentation-Ready)

```
useEffect(() => {  
  // side effect  
}, [dependencies]);
```

Parts:

- **Effect function** → what to run
- **Dependency array** → when to run

4. Dependency Array—MOST IMPORTANT INTERVIEW AREA

Case 1: No dependency array

```
useEffect(() => {  
  console.log("Runs on every render");  
});
```

Runs:

- Initial render
- Every re-render

Case 2: Empty dependency array

```
useEffect(() => {  
  console.log("Runs once");  
}, []);
```

Runs:

- Only on mount

Interview line:

“Equivalent to componentDidMount.”

Case 3: With dependencies

```
useEffect(() => {  
  console.log("Runs when count changes");  
}, [count]);
```

Runs:

- On mount
- Whenever `count` changes

5. Hands-On #1 (You Do This)

Task:

Create a counter using `useState`.

Add a `useEffect` that logs “**Count changed**” whenever the count updates.

Before coding, answer this **one question**:

👉 **What should go in the dependency array if the effect depends on `count`?**

(Answer in plain words or code—your choice.)

6. Cleanup Function (High-Frequency Interview Topic)

```
useEffect(() => {  
  const id = setInterval(() => {  
    console.log("tick");  
  }, 1000);  
  
  return () => {  
    clearInterval(id);  
  };  
}, []);
```

Interview explanation:

The cleanup function runs before the effect re-runs and when the component unmounts.

Used for:

- Clearing timers
- Removing event listeners
- Cancelling subscriptions

7. Common Interview Traps

✗ Updating state unconditionally

```
useEffect(() => {  
  setCount(count + 1);  
});
```

Why this is bad:

- Infinite re-render loop

✗ Missing dependencies

```
useEffect(() => {  
  console.log(count);  
}, []);
```

Interview answer:

This causes stale closures and incorrect behavior.

8. useEffect vs Event Handlers (Important Distinction)

✗ Wrong mental model:

“useEffect runs on button click”

✓ Correct:

“useEffect reacts to state changes, not events”

9. Documentation-Ready Summary

`useEffect` is used to run side effects after render. It accepts a dependency array that controls when the effect runs. Effects can return a cleanup function to release resources. Incorrect dependencies can cause stale data or infinite loops.

10. Hands-On Assignment (Do One at a Time)

Task A: Title Updater

- State: `count`
- Effect: update `document.title` when count changes

Task B: Timer

- Start a timer on the mount.
- Cleanup on unmount

useRef

1. Interview Definition (Memorize This)

`useRef` returns a mutable object whose `.current` property persists across renders without causing re-renders when it changes.

Key interview contrast:

“`useRef` stores values without triggering a re-render; `useState` stores values and triggers re-renders.”

2. Why `useRef` Exists (Conceptual Model)

`useRef` solves **two distinct problems**:

Problem A: Accessing DOM elements

- Input focus
- Measuring size
- Scrolling

Problem B: Persisting values across renders

- Previous values
- Timers
- Flags

Mental model:

- `useState` → causes re-render
- `useRef` → survives re-render, does NOT cause re-render

3. Basic Syntax (Documentation-Ready)

```
const myRef = useRef(initialValue);
```

Access:

```
myRef.current
```

4. DOM Access Example (High-Frequency Interview Topic)

```
function InputFocus() {  
  const inputRef = useRef(null);  
  
  const focusInput = () => {  
    inputRef.current.focus();  
  };  
  
  return (  
    <>  
      <input ref={inputRef} />  
      <button onClick={focusInput}>Focus</button>  
    </>  
  );  
}
```

Interview explanation:

React assigns the DOM node to `ref.current` after the component mounts.

5. Hands-On #1 (You Do This)

Task:

Create an input field and a button.

When the button is clicked, the input should:

- Receive focus
- Be cleared

Before coding, answer **one question**:

👉 **Why does changing `ref.current` NOT trigger a re-render?**

(Short explanation is enough.)

6. Persisting Values Across Renders (Advanced Use)

```
const renderCount = useRef(0);
```

```
useEffect(() => {  
  renderCount.current += 1;  
});
```

Why this works:

- Value persists
- No re-render loop

7. Interview Trap: useRef vs useState

❌ Wrong

```
const [timerId, setTimerId] = useState(null);
```

✅ Correct

```
const timerId = useRef(null);
```

Interview line:

`useRef` is ideal for storing non-UI data like timers and mutable instance variables.

8. useRef vs Variables (Very Common Question)

❌ Local variable

```
let count = 0;
```

Why wrong:

- Resets on every render

✓ useRef

```
const count = useRef(0);
```

9. Documentation-Ready Summary

`useRef` provides a mutable container that persists across renders without causing re-renders. It is commonly used for DOM access and storing mutable values such as timers or previous state. Updating `.current` does not trigger rendering.

10. Hands-On Assignment (Choose One)

Task A: Previous Value Tracker

- Track previous `count`
- Display current vs previous

Task B: Render Counter

- Count how many times the component renders

useLayoutEffect

1. Interview Definition (Exact Wording)

`useLayoutEffect` runs synchronously after all DOM mutations but before the browser paints the screen.

Key contrast phrase:

“`useEffect` runs after paint; `useLayoutEffect` runs before paint.”

2. Where `useLayoutEffect` Fits in the Lifecycle

Render timeline (critical for interviews):

1. React renders JSX (virtual DOM)
2. React commits changes to the real DOM
3. **`useLayoutEffect` runs (blocking)**

4. Browser paints the screen
5. `useEffect` runs (non-blocking)

3. Why `useLayoutEffect` Exists

Problem with `useEffect`:

- UI may briefly paint in the wrong state
- Measurements may be inaccurate

`useLayoutEffect` is used when you must:

- Measure DOM size/position
- Apply synchronous layout changes
- Prevent visible flicker

4. Syntax (Documentation-Ready)

```
useLayoutEffect(() => {  
  // DOM reads / writes  
}, [dependencies]);
```

Same API as `useEffect`, different timing.

5. Example: Preventing Flicker (Classic Interview Example)

```
useLayoutEffect(() => {  
  const { height } = ref.current.getBoundingClientRect();  
  setHeight(height);  
}, []);
```

Why not `useEffect`?

Because the browser would paint first, causing a visible jump.

6. Hands-On #1 (Think Before Coding)

Scenario:

You need to:

- Measure an element's width
- Set another element's position based on it
- Ensure the user never sees incorrect layout

👉 Question:

Should this logic go in `useEffect` or `useLayoutEffect`, and why?

(Answer in one or two sentences.)

7. Interview Warning (Very Important)

```
useLayoutEffect(() => {  
  
  setState(...);  
  
});
```

Why this is dangerous:

- Blocks painting
- Can cause performance issues

Interview line:

`useLayoutEffect` should be used sparingly because it blocks the browser from painting.

8. useEffect vs useLayoutEffect (Table for Memory)

Aspect	useEffect	useLayoutEffect
Runs	After paint	Before paint
Blocking	No	Yes
Use case	Data fetching, subscriptions	DOM measurement

Performance	Safer	Risky if misused
-------------	-------	------------------

9. Documentation-Ready Summary

`useLayoutEffect` is identical to `useEffect` in API but differs in timing. It runs synchronously after DOM updates and before paint, making it suitable for layout measurements and preventing UI flicker. Because it blocks painting, it should be used only when necessary.

10. Hands-On Assignment (Choose One)

Task A: Width Measurement

- Measure an element's width on mount
- Display the width
- Ensure no flicker

Task B: Tooltip Position

- Position a tooltip based on target element size
- Prevent visible jumps

`useContext`

1. Interview Definition (Memorize)

`useContext` allows a component to consume values from a React Context without prop drilling.

High-signal interview phrase:

“`useContext` provides access to shared state across the component tree.”

2. The Problem `useContext` Solves

Prop drilling problem:

- Data must pass through multiple intermediate components
- Intermediate components do not use the data

`useContext` solves this by:

- Providing a **global-ish** store scoped to a subtree

3. Core Pieces (Must Know for Interviews)

There are **three required steps**:

1. Create context
2. Provide context
3. Consume context

4. Step 1: Create Context

```
const ThemeContext = React.createContext();
```

Interview note:

The default value is only used when no provider exists.

5. Step 2: Provide Context

```
<ThemeContext.Provider value="dark">  
  
  <App />  
  
</ThemeContext.Provider>
```

Key point:

- `value` can be **any JS value**
- Objects cause re-renders if recreated

6. Step 3: Consume Context

```
const theme = useContext(ThemeContext);
```

Important rule:

`useContext` must be used inside a matching Provider.

7. Hands-On #1 (Architecture Thinking)

Scenario:

You have a `UserContext` that stores `{ name, role }`.

👉 Question:

Where should the `UserContext.Provider` be placed in the component tree, and why?

(One concise explanation.)

8. Interview Trap: Context Causes Re-renders

If provider value changes:

- **ALL consuming components re-render**

```
<UserContext.Provider value={{ user }}>
```

Why this is risky:

- New object reference on every render

Interview line:

Context is not a performance optimization tool.

9. Context vs Props (Very Common Question)

Aspect	Props	Context
Scope	Parent → child	Tree-wide
Explicit	Yes	Implicit
Best for	Local data	Global/shared data

10. Context vs State Management Libraries

When to use `useContext`:

- Theme
- Auth user
- Language
- Feature flags

When NOT to:

- Frequently changing data
- Large-scale app state

11. Documentation-Ready Summary

`useContext` enables components to access shared data without prop drilling. It requires a Context object, a Provider, and consumers. Context updates trigger re-renders in all consumers and should be used for low-frequency, global data.

12. Hands-On Assignment (Choose One)

Task A: Theme Context

- Light / Dark mode
- Toggle from a child component

Task B: Auth Context

- Store `isLoggedIn`
- Conditional rendering

useReducer

1. Interview Definition (Memorize This)

`useReducer` is a hook used to manage complex state logic through explicit state transitions using actions.

High-value interview phrase:

“`useReducer` is preferred over `useState` when state logic becomes complex or when the next state depends on the previous state.”

2. Why `useReducer` Exists

Problems with `useState`:

- Many related state variables
- Conditional updates
- Hard-to-follow logic

`useReducer` introduces:

- Centralized logic
- Predictable state transitions
- Easier debugging

3. Core Concepts (Must Know)

A reducer has **three parts**:

1. **State**
2. **Action**
3. **Reducer function**

4. Basic Syntax (Documentation-Ready)

```
const [state, dispatch] = useReducer(reducer, initialState);
```

Reducer:

```
function reducer(state, action) {  
  switch (action.type) {  
    case 'increment':  
      return { count: state.count + 1 };  
    default:  
      return state;  
  }  
}
```

5. Interview Mental Model

- `dispatch(action)` → intent
- `reducer` → decision logic
- `state` → result

Interview phrase:

Reducers must be pure functions.

6. Hands-On #1 (Think Before Coding)

Scenario:

You are building a form with:

- Multiple inputs
- Validation rules
- Reset logic

👉 Question:

Why is `useReducer` a better fit than multiple `useState` hooks here?

(One or two clear reasons.)

7. Interview Trap: Mutating State

✗ Wrong

```
state.count++;
```

```
return state;
```

✓ Correct

```
return { ...state, count: state.count + 1 };
```

Why:

Reducers must return new state objects.

8. Lazy Initialization (Advanced Interview Topic)

```
useReducer(reducer, initialArg, init);
```

Used when:

- Initial state is expensive to compute

9. `useReducer` + `useContext` (Very Common Pair)

Pattern:

- `useReducer` → logic
- `useContext` → sharing state

Interview line:

This pattern mimics Redux at a smaller scale.

10. Documentation-Ready Summary

`useReducer` manages complex state through reducers and actions. It centralizes update logic, ensures predictability, and is ideal for scenarios involving multiple related state updates or conditional transitions.

11. Hands-On Assignment (Choose One)

Task A: Counter with Reducer

- increment
- decrement
- reset

Task B: Form State

- Multiple fields
- Single reducer

`useMemo`

1. Interview Definition (Use This Wording)

`useMemo` memoizes the result of a computation so it is only recalculated when its dependencies change.

High-signal interview phrase:

“`useMemo` is a performance optimization hook, not a correctness tool.”

2. Why `useMemo` Exists

Problem:

- Every render re-executes component code
- Expensive calculations can slow rendering

`useMemo`:

- Caches a computed value
- Avoids unnecessary recalculation

Mental model:

“Remember the value unless dependencies change.”

3. Basic Syntax (Documentation-Ready)

```
const memoizedValue = useMemo(() => {  
  return expensiveCalculation(a, b);  
}, [a, b]);
```

Parts:

- Function → computation
- Dependency array → cache invalidation

4. What `useMemo` Does NOT Do (Interview Trap)

✗ It does NOT:

- Prevent re-renders
- Replace `useEffect`
- Automatically improve performance

Interview line:

Using `useMemo` everywhere can hurt readability without improving performance.

5. Common Interview Example

```
const filteredList = useMemo(() => {  
  return items.filter(item => item.active);  
}, [items]);
```

Why this helps:

- Filtering runs only when `items` changes

6. Hands-On #1 (Think Before Coding)

Scenario:

You have:

- A large list
- A search input
- Typing causes re-renders

👉 **Question:**

Which value should be wrapped in `useMemo`:

- the input state
- or the filtered list—and **why**?

(Answer in one or two sentences.)

7. Dependency Array Rules (High-Probability Interview Area)

You must include:

- Every value used inside the memo callback
- That comes from component scope

Interview explanation:

Missing dependencies lead to stale memoized values.

8. useMemo vs Normal Variable

 Without `useMemo`

```
const result = heavyCalculation(data);
```

Runs:

- On every render

 With `useMemo`

```
const result = useMemo(() => heavyCalculation(data), [data]);
```

Runs:

- Only when `data` changes

9. Documentation-Ready Summary

`useMemo` memoizes computed values to avoid expensive recalculations on every render. It should be used selectively for performance optimization and requires accurate dependency tracking to avoid stale values.

10. Hands-On Assignment (Choose One)

Task A: Expensive Calculation

- Create a slow function (loop or computation)
- Memoize its result
- Trigger re-renders without recalculating

Task B: Filtered List

- Input field
- Large array
- Memoize filtered results

useCallback

1. Interview Definition (Use This Exactly)

useCallback memoizes a function so that the same function reference is preserved between renders unless its dependencies change.

High-signal interview phrase:

“useCallback is used to prevent unnecessary re-renders caused by changing function references.”

2. Why useCallback Exists

Key React fact:

- Functions are **recreated on every render**

This becomes a problem when:

- Functions are passed as props
- Child components are memoized (`React.memo`)

useCallback:

- Preserves function identity
- Prevents avoidable child re-renders

3. Basic Syntax (Documentation-Ready)

```
const handleClick = useCallback(() => {  
  setCount(c => c + 1);  
}, []);
```

Mental model:

“Remember this function unless dependencies change.”

4. useCallback vs useMemo (Common Interview Question)

```
useCallback(fn, deps)
```

```
useMemo(() => fn, deps)
```

Interview answer:

`useCallback` memoizes functions; `useMemo` memoizes values.

5. When `useCallback` Actually Helps (Important)

It helps when **all three** are true:

1. Function is passed to a child
2. Child is memoized
3. Re-renders are frequent or expensive

Otherwise:

`useCallback` adds overhead with no benefit.

6. Hands-On #1 (Think Before Coding)

Scenario:

You have:

- Parent component with state
- Child wrapped in `React.memo`
- Parent passes a handler function

👉 **Question:**

Why does the child still re-render without `useCallback`?

(Answer in one clear sentence.)

7. Dependency Array Rules (Interview Trap)

```
useCallback(() => {  
  console.log(count);  
}, []);
```

Why this is wrong:

- `count` becomes stale

Correct:

```
useCallback(() => {  
  console.log(count);  
}, [count]);
```

8. Common Overuse Mistake

✗ Wrapping every function

```
const fn = useCallback(() => {}, []);
```

Interview line:

Premature optimization can reduce readability without improving performance.

9. Documentation-Ready Summary

`useCallback` memoizes function references to prevent unnecessary re-renders, especially when passing callbacks to memoized child components. It should be used selectively and with correct dependencies to avoid stale closures.

10. Hands-On Assignment (Choose One)

Task A: Parent–Child Re-render

- Parent has counter
- Child is memoized
- Prevent unnecessary child re-renders

Task B: List Item Handler

- Memoized list items
- Stable click handlers

`useInsertionEffect`

1. Interview Definition (Precise Wording)

`useInsertionEffect` runs synchronously before any DOM mutations are applied, primarily for injecting styles.

High-signal interview phrase:

“`useInsertionEffect` is intended for CSS-in-JS libraries to insert styles before layout and paint.”

2. Where It Fits in the Lifecycle (Critical)

Execution order:

1. Render phase
2. **`useInsertionEffect`**
3. DOM mutations
4. `useLayoutEffect`
5. Browser paint
6. `useEffect`

Key comparison:

- `useInsertionEffect` → **before DOM mutations**
- `useLayoutEffect` → after DOM mutations, before paint

3. Why `useInsertionEffect` Exists

Problem:

- CSS-in-JS libraries need to inject styles
- Styles must exist **before layout calculations**

If styles are injected too late:

- Layout shift
- Incorrect measurements
- Visual flicker

`useInsertionEffect` solves this timing problem.

4. Syntax (Documentation-Ready)

```
useInsertionEffect(() => {  
  
  // inject styles
```

```
}, [dependencies]);
```

Same signature as other effects, **different timing**.

5. VERY IMPORTANT Warning (Interview Gold)

```
useInsertionEffect(() => {  
  setState(...);  
});
```

✗ This is **not allowed**.

Interview explanation:

State updates inside `useInsertionEffect` are forbidden because it runs during React's render cycle.

6. When You Should NOT Use It

You should NOT use `useInsertionEffect` for:

- Data fetching
- DOM measurement
- Event listeners
- Application logic

Interview line:

Most application developers will never need `useInsertionEffect`.

7. Hands-On (Conceptual, Not Coding)

Scenario:

You are building a CSS-in-JS library that:

- Generates class names dynamically
- Injects styles into `<style>` tags

👉 Question:

Why would `useLayoutEffect` be too late for this job?

(Answer in one or two sentences.)

8. `useInsertionEffect` vs `useLayoutEffect` vs `useEffect` (Memory Table)

Hook	Runs	Use case
<code>useInsertionEffect</code>	Before DOM mutation	Style injection
<code>useLayoutEffect</code>	After DOM mutation, before paint	Layout measurement
<code>useEffect</code>	After paint	Side effects

9. Documentation-Ready Summary

`useInsertionEffect` is a specialized hook designed for style injection in CSS-in-JS libraries. It runs before DOM mutations, ensuring styles are available for layout calculations. It should not be used for state updates or application logic.

10. Interview Guidance (What to Say)

If asked:

“Have you used `useInsertionEffect`?”

Best professional answer:

“I understand its purpose and timing, but it’s mainly for library authors. In application code, I rely on `useEffect` and `useLayoutEffect`.”

`useQuery`

1. Interview Definition (Say This Clearly)

`useQuery` is a hook for managing server state, including fetching, caching, synchronization, and background updates.

High-signal interview phrase:

“`useQuery` manages server state, not client UI state.”

2. Why `useQuery` Exists (Very Important)

Problem with `useEffect` + `useState`:

- Manual loading states
- Manual error handling
- No caching
- Re-fetching logic everywhere

`useQuery` provides:

- Automatic caching
- Background refetching
- Request deduplication
- Built-in loading and error states

3. Server State vs Client State (Interview Favorite)

Client state (`useState` / `useReducer`):

- UI state
- Modals, toggles, form inputs

Server state (`useQuery`):

- Data from APIs
- Shared across components
- Can become stale

Interview line:

Server state is asynchronous, shared, and time-sensitive.

4. Basic Syntax (Documentation-Ready)

```
const { data, isLoading, error } = useQuery({  
  queryKey: ['users'],  
  queryFn: fetchUsers  
});
```

Key parts:

- `queryKey` → cache identity
- `queryFn` → async fetcher
- Returned state → managed automatically

5. Interview Trap: `queryKey`

```
queryKey: ['user', userId]
```

Why this matters:

- Cache is scoped per key
- Different `userId` → different cache entry

Interview phrase:

The query key uniquely identifies cached data.

6. Loading & Error Handling (Cleaner Than useEffect)

Instead of manual flags:

```
if (isLoading) return 'Loading...';  
  
if (error) return 'Error';
```

Interview insight:

`useQuery` removes boilerplate from async state management.

7. When NOT to Use useQuery

Do NOT use `useQuery` for:

- Form input state
- UI toggles
- Derived calculations

Interview line:

`useQuery` replaces `useEffect` only for data fetching—not general side effects.

8. Hands-On (Conceptual, Not Coding)

Scenario:

Two components request the same API data using the same `queryKey`.

👉 Question:

What happens under the hood, and why is this beneficial?

(Answer in one or two sentences.)

9. useQuery vs Redux (Common Comparison)

Interview-safe answer:

Redux manages client state; `useQuery` manages server state. They solve different problems and often work together.

10. Documentation-Ready Summary

`useQuery` is a server-state management hook from TanStack Query. It handles data fetching, caching, synchronization, and background updates. It replaces `useEffect`-based fetching logic and should only be used for asynchronous server data.

11. Final Interview Wrap-Up (Very Important)

If asked:

“Which hooks do you use for data fetching?”

Best answer:

“I use `useQuery` for server data and reserve `useEffect` for non-data side effects.”

Controlled vs Uncontrolled Components.

High-Level Definition

- **Controlled components:** React **controls the form data** via state.
- **Uncontrolled components:** The **DOM controls the form data**, React reads it when needed.

This distinction applies mainly to **form inputs**.

Controlled Components

What They Are

A **controlled component** stores its value in **React state** and updates it via `onChange`.

```
function ControlledInput() {
  const [value, setValue] = useState("");

  return (
    <input
      value={value}
      onChange={e => setValue(e.target.value)}
    />
  );
}
```

Characteristics

- Single source of truth (React state)
- Predictable and debuggable
- Easy validation and formatting
- Re-render on every change

When to Use Controlled Components

- Validation is required
- Input value affects UI
- Dynamic enable/disable logic
- Complex forms

Uncontrolled Components

What They Are

An **uncontrolled component** lets the browser manage the input value.

```
function UncontrolledInput() {  
  const inputRef = useRef(null);  
  
  function handleSubmit() {  
    console.log(inputRef.current.value);  
  }  
  
  return <input ref={inputRef} />;  
}
```

Characteristics

- No state updates on input change
- Less React code
- DOM is the source of truth
- Read value only when needed

When to Use Uncontrolled Components

- Simple forms
- One-time value access
- Integrating with non-React code
- Performance-sensitive cases

Key Differences

Aspect	Controlled	Uncontrolled
--------	------------	--------------

Source of truth	React state	DOM
Updates	<code>onChange</code>	Direct DOM
Validation	Easy	Manual
Performance	More renders	Fewer renders
Predictability	High	Lower

Mixed Pattern

```
<input defaultValue="Hello" />
```

- Uncontrolled initially
- Cannot later switch to controlled

Never switch between controlled and uncontrolled during a component's lifecycle.

Common Mistakes

Switching Control Mode

```
<input value={value || ""} />
```

If `value` becomes `undefined`, React warns.

Using Refs Instead of State for UI Data

If the value affects rendering, it should be state.

One-Sentence Rule

If React needs to know about it while the user types, make it controlled.

If React only needs the value at submit time, uncontrolled is acceptable.

Controlled components trade performance for predictability; uncontrolled components trade predictability for simplicity.

React generally recommends **controlled components**, especially for complex forms.

Below is a **clear, practical explanation of RWD (Responsive Web Design)**, focused on *what it is, why it matters, and how it is implemented in modern web development*.

Responsive Web Design

What Is RWD?

Responsive Web Design is an approach to building websites that **adapt their layout and content to different screen sizes and devices** using the same codebase.

Goal:

One website, optimal experience on mobile, tablet, and desktop.

Why RWD Matters

- Majority of traffic is mobile
- Improves usability and accessibility
- Better SEO (Google prioritizes mobile-friendly sites)
- Easier maintenance than separate mobile/desktop sites

The Three Core Pillars of RWD

1. Fluid Layouts (Flexible Grids)

Layouts are based on **relative units**, not fixed pixels.

```
.container {  
  width: 100%;  
  max-width: 1200px;  
}
```

Common units:

- %
- vw, vh
- rem, em

This allows content to resize naturally.

2. Media Queries (Breakpoints)

Media queries apply styles **based on screen size or device characteristics**.

```
@media (max-width: 768px) {  
  .sidebar {  
    display: none;  
  }  
}
```

Typical breakpoints (guidelines, not rules):

- Mobile: $\leq 768\text{px}$
- Tablet: $769\text{px} - 1024\text{px}$
- Desktop: $\geq 1024\text{px}$

3. Flexible Media (Images & Videos)

Media should scale within their containers.

```
img {  
  max-width: 100%;  
  height: auto;  
}
```

Prevents overflow and layout breaking on small screens.

Mobile-First Design (Best Practice)

Mobile-first means:

- Design for small screens first
- Add enhancements for larger screens

```
.card {  
  padding: 1rem;  
}
```

```
@media (min-width: 1024px) {  
  .card {  
    padding: 2rem;  
  }  
}
```

Benefits:

- Better performance
- Clear content prioritization
- Easier scalability

RWD in Modern Frameworks

RWD is handled mostly with **CSS**, not React logic.

Common approaches:

- CSS media queries
- Flexbox and Grid

- Utility-first frameworks (Tailwind CSS)
- Component-level responsiveness

```
<div className="grid grid-cols-1 md:grid-cols-3">
```

React handles **structure**, CSS handles **responsiveness**.

Flexbox vs Grid (Quick Guidance)

Use Case	Prefer
One-dimensional layout	Flexbox
Two-dimensional layout	Grid
Page-level structure	Grid
Component alignment	Flexbox

Common RWD Mistakes

- Fixed widths (`width: 600px`)
- Designing desktop-first only
- Hiding critical content on mobile
- Too many breakpoints
- Ignoring touch targets

One-Line Rule

RWD is not about specific devices—it's about adapting to available space.

Responsive Web Design ensures your UI adapts to any screen using flexible layouts, media queries, and scalable content.

Tailwind CSS

Tailwind CSS is a **utility-first CSS framework**.

Instead of writing custom CSS classes, you **compose styles directly in your markup** using small, single-purpose utility classes.

```
<button class="bg-blue-500 text-white px-4 py-2 rounded">  
  Save
```

</button>

You style UI by **combining utilities**, not by writing new CSS rules.

Why Tailwind CSS Is Popular

Key Benefits

- No context switching between CSS and JSX
- Highly consistent design system
- No unused CSS in production
- Faster development once learned
- Easy responsive and dark-mode styling

Utility-First vs Traditional CSS

Traditional CSS	Tailwind CSS
Write custom classes	Use predefined utilities
CSS files grow	CSS stays small
Naming is hard	No naming required
Context switching	Styles live with markup

Installing Tailwind CSS

This is the **most common modern setup**.

1. Install Tailwind and Dependencies

```
npm install tailwindcss @tailwindcss/vite
```

2. Configure Tailwind Paths

Edit **vite.config.ts**:

```
import { defineConfig } from 'vite'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [
    tailwindcss(),
  ],
})
```

3. Import Tailwind to CSS

In `src/index.css` (or `main.css`):

```
@import "tailwindcss";
```

4. Start the Dev Server

```
npm run dev
```

Tailwind is now installed and ready.

5. Using Tailwind in React

```
function Button() {  
  return (  
    <button className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700">  
      Click me  
    </button>  
  );  
}
```

Responsive Design in Tailwind

Tailwind uses **mobile-first breakpoints**.

```
<div class="p-4 md:p-8 lg:p-12">
```

Prefix	Meaning
--------	---------

<code>sm:</code>	≥ 640px
------------------	---------

<code>md:</code>	≥ 768px
------------------	---------

<code>lg:</code>	≥ 1024px
------------------	----------

<code>xl:</code>	≥ 1280px
------------------	----------

Dark Mode

Enable in `tailwind.config.js`:

```
darkMode: "class",
```

Usage:

```
<div class="bg-white dark:bg-gray-900 text-black dark:text-white">
```

Customizing Tailwind (Design System)

```
theme: {  
  extend: {  
    colors: {  
      brand: "#1e40af",  
    },  
  },  
},
```

```
<div class="text-brand">
```

How Tailwind Works Internally

1. Scans your files for class names
2. Generates **only the CSS you use**
3. Removes unused styles in production
4. Outputs a very small CSS file

This is why Tailwind scales well.

Common Tailwind Utilities You'll Use Daily

- Layout: `flex`, `grid`, `gap-*`
- Spacing: `p-*`, `m-*`
- Typography: `text-*`, `font-*`
- Colors: `bg-*`, `text-*`
- Borders: `rounded-*`, `border-*`
- Effects: `shadow-*`, `hover:*`

Best Practices

- Extract reusable UI into components
- Use `clsx` or `classnames` for conditional styles
- Avoid overly long class strings in one element
- Let Tailwind handle layout; React handles logic

When Tailwind May Not Be Ideal

- Very small static sites
- Teams unfamiliar with utility-first CSS

- Strict separation of markup and styling required

One-Sentence Rule

Tailwind is a design system expressed as CSS utilities, not a component library.

Tailwind CSS lets you build responsive, consistent UIs faster by composing styles directly in your markup instead of writing custom CSS.

RWD using Tailwind

How Responsive Design Works in Tailwind

Tailwind is **mobile-first**.

That means:

default → mobile
sm: → ≥ 640px
md: → ≥ 768px
lg: → ≥ 1024px
xl: → ≥ 1280px
2xl: → ≥ 1536px

Example:

grid-cols-1 md:grid-cols-2

- Mobile: 1 column
- Tablet/Desktop: 2 columns

Common Tailwind RWD Classes

These are the **most-used responsive utilities in real projects**.

◆ Layout

flex
grid
grid-cols-1 md:grid-cols-2
flex-col md:flex-row

hidden md:flex
block md:hidden

◆ Spacing

p-4 sm:p-6 md:p-10
px-4 sm:px-6
gap-4 md:gap-6
space-y-4 md:space-y-6

◆ Width / Height

w-full
max-w-md
max-w-4xl
min-h-screen
h-screen (avoid on mobile)

◆ Typography

text-sm md:text-base
text-xl md:text-2xl
text-3xl font-bold

◆ Alignment

items-center
justify-center
text-center md:text-left

◆ Visibility

hidden md:block
hidden md:flex

◆ Images

w-32 sm:w-40 md:w-48
object-cover

◆ Forms

w-full
rounded-full
focus:ring-2

Final Cheat Sheet

RWD in Tailwind = Layout switches + max-width + spacing control

If you master just these **5 patterns**, you're 90% there:

1. `grid-cols-1 md:grid-cols-2`
2. `hidden md:flex`
3. `w-full max-w-*`
4. `p-* sm:p-*`
5. `min-h-screen`

`min-h-screen`

`w-full max-w-md`

`grid-cols-1 md:grid-cols-2`

`hidden md:flex`

`p-4 sm:p-6 md:p-10`

`gap-4 md:gap-6`

`text-center md:text-left`

What Is a `@media` Query?

A **media query** allows you to **apply CSS conditionally** based on:

- Screen size
- Device characteristics
- Viewport features

It is the core mechanism behind **Responsive Web Design (RWD)**.

```
@media (max-width: 768px) {  
  body {  
    background: lightgray;  
  }  
}
```

Why Media Queries

They let your layout:

- Adapt to different screen sizes
- Optimize readability and usability
- Support multiple devices with one codebase

Basic Syntax

```
@media media-type and (condition) {  
  /* CSS rules */  
}
```

Common media types:

- `screen` (default)
- `print`
- `all`

Most modern usage omits the media type.

Most Common Media Features

Viewport Width (Most Used)

```
@media (max-width: 768px) { }  
@media (min-width: 1024px) { }
```

- `max-width` → applies **up to** a size
- `min-width` → applies **from** a size upward

Orientation

```
@media (orientation: landscape) {  
  /* styles */  
}
```

Resolution (Advanced)

```
@media (min-resolution: 2dppx) {  
  /* retina displays */  
}
```

Mobile-First vs Desktop-First

Mobile-First (Recommended)

```
.card {  
  padding: 1rem;  
}
```

```
@media (min-width: 768px) {
```

```
.card {  
  padding: 2rem;  
}
```

- Base styles = mobile
- Add complexity as screen grows

Desktop-First

```
@media (max-width: 768px) {  
  .sidebar {  
    display: none;  
  }  
}
```

Used less often today.

Common Breakpoints

Device	Width
Mobile	$\leq 768\text{px}$
Tablet	769–1024px
Desktop	$\geq 1024\text{px}$

Avoid device-specific thinking—design for **content**, not devices.

Combining Conditions

```
@media (min-width: 768px) and (max-width: 1024px) {  
  /* tablet only */  
}
```

Media Queries in Tailwind (Mapping)

```
<div class="p-4 md:p-8 lg:p-12">
```

Equivalent CSS:

```
@media (min-width: 768px) { }  
@media (min-width: 1024px) { }
```

Media Queries vs Container Queries

- **Media queries** → based on viewport
- **Container queries** → based on parent size (newer)

Use media queries for **page layout**, container queries for **components**.

Common Mistakes

- Too many breakpoints
- Designing for devices instead of content
- Fixed widths
- Hiding content instead of adapting it

One-Liner

Media queries ask the browser “When should these styles apply?”

Use media queries to adapt layouts based on available space, not specific devices.

Routing

Routing is an essential concept in Single Page Applications (SPA). When your application is divided into separated logical sections, and all of them are under their own URL, your users can easily share links among each other.

Routing allows a React application to **render different components based on the URL**, without reloading the page.

This enables **Single Page Applications (SPAs)**.

Example:

- / → Home
- /login → Login
- /products/42 → Product Details

Why Router

React by itself:

- Does **not** handle URLs
- Does **not** manage navigation

A routing library:

- Listens to URL changes
- Decides which component to render
- Updates the UI without a full page reload

React Router: Core Concepts

React Router provides these key pieces:

- **BrowserRouter** – enables routing
- **Routes** – route container
- **Route** – maps URL → component
- **Link / NavLink** – navigation
- Hooks (**useParams**, **useNavigate**, etc.)

Basic Setup

1. Install React Router

```
npm install react-router-dom
```

2. Wrap Your App with **BrowserRouter**

```
import { BrowserRouter } from "react-router-dom";
```

```
ReactDOM.createRoot(document.getElementById("root")).render(  
  <BrowserRouter>  
    <App />  
  </BrowserRouter>  
);
```

This enables routing for the entire app.

Defining Routes

```
import { Routes, Route } from "react-router-dom";
```

```
function App() {  
  return (  
    <Routes>  
      <Route path="/" element={<Home />} />  
      <Route path="/login" element={<Login />} />  
      <Route path="/about" element={<About />} />  
    </Routes>  
  );  
}
```

Key Rules

- `Routes` replaces `Switch` (v6)
- `element` receives JSX, not a component reference
- Routes are matched **exactly by default**

Navigation with `Link`

```
import { Link } from "react-router-dom";
```

```
<Link to="/login">Login</Link>
```

Why not `<a>`?

- `<a>` reloads the page
- `Link` updates the URL client-side

Dynamic Routes

```
<Route path="/users/:id" element={<User />} />
```

```
import { useParams } from "react-router-dom";
```

```
function User() {  
  const { id } = useParams();  
  return <h1>User ID: {id}</h1>;  
}
```

Used for:

- Detail pages
- Resource IDs
- Slugs

Nested Routes

Used for layouts and shared UI.

```
<Route path="/dashboard" element={<DashboardLayout />>  
  <Route index element={<DashboardHome />} />  
  <Route path="settings" element={<Settings />} />  
</Route>
```

```
import { Outlet } from "react-router-dom";
```

```
function DashboardLayout() {
  return (
    <>
      <Sidebar />
      <Outlet />
    </>
  );
}
```

Programmatic Navigation

```
import { useNavigate } from "react-router-dom";
```

```
const navigate = useNavigate();
```

```
navigate("/login");
```

Used after:

- Form submission
- Authentication
- Conditional redirects

Route Protection

```
function ProtectedRoute({ children }) {
  return isAuthenticated ? children : <Navigate to="/login" />;
}
```

```
<Route
  path="/profile"
  element={
    <ProtectedRoute>
      <Profile />
    </ProtectedRoute>
  }
/>
```

404 / Not Found Route

```
<Route path="*" element={<NotFound />} />
```

Always place it last.

URL State vs React State

Routes are:

- **Global**
- **Shareable**
- **Persistent on refresh**

Use routes for:

- Pages
- App state that should be bookmarkable

One-Sentence Rule

Routing is conditional rendering based on the URL, not page navigation.

React stays mounted; only components change.

Routes map URLs to components, enabling SPA navigation without full page reloads.

Common Mistakes

- Using `<a>` instead of `<Link>`
- Forgetting `BrowserRouter`
- Overusing routes for local UI state
- Mixing routing logic into components